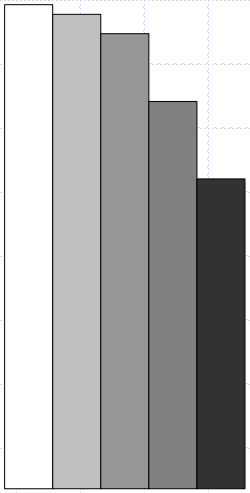
Paralelni programski modeli

- pthreads
- MPI
- Cilk/OpenMP
- IPP, TBB

- Model sa deljenom memorijom
- Model sa slanjem poruka

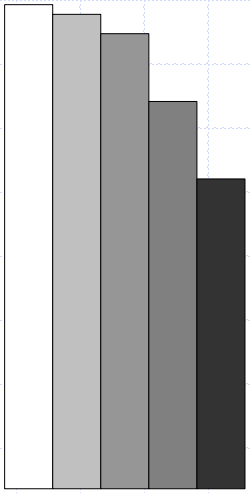
Računanje Pi sa integracijom (MPI)

```
static long num_steps = 100000;
void main(int argc, char* argv[])
{
   int i_start, i_end, i, myid, numprocs;
   double pi, mypi, x, step, sum = 0.0;
   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
   MPI_Comm_rank(MPI_COMM_WORLD, &myid);
   MPI_BCAST(&num_steps, 1, MPI_INT, 0,
                        MPI_COMM_WORLD);
   i_start = myid * (num_steps/numprocs)
   i_end = i_start + (num_steps/numprocs)
   step = 1.0 / (double) num_steps;
   for (i = i_start; i < i_end; i++) {
      x = (i + 0.5) * step
      sum = sum + 4.0 / (1.0 + x*x);
   }
   mypi = step * sum;
   MPI_REDUCE(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM,
                           0, MPI_COMM_WORLD);
   if (myid == 0)
      printf("Pi = %f\n", pi);
   MPI_Finalize();
}
```



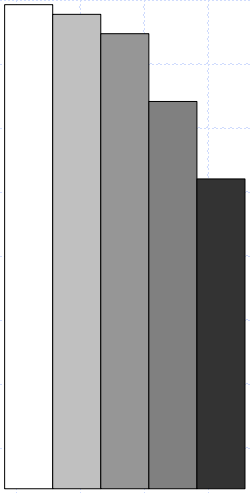
Računanje Pi sa integracijom (MPI)

```
static long num_steps = 100000;
void main(int argc, char* argv[])
{
   int i_start, i_end, i, myid, numprocs;
   double pi, mypi, x, step, sum = 0.0;
   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
   MPI_Comm_rank(MPI_COMM_WORLD, &myid);
   MPI_BCAST(&num_steps, 1, MPI_INT, 0,
                        MPI_COMM_WORLD);
   i_start = myid * (num_steps/numprocs)
   i_end = i_start + (num_steps/numprocs)
   step = 1.0 / (double) num_steps;
   for (i = i_start; i < i_end; i++) {
      x = (i + 0.5) * step
      sum = sum + 4.0 / (1.0 + x*x);
   }
   mypi = step * sum;
   MPI_REDUCE(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM,
                          0, MPI_COMM_WORLD);
   if (myid == 0)
      printf("Pi = %f\n", pi);
   MPI_Finalize();
}
```



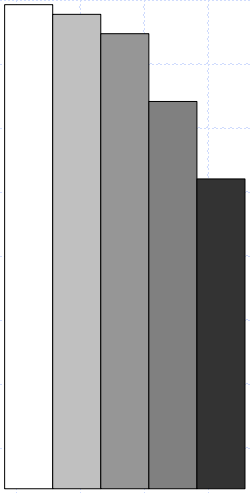
Računanje Pi sa integracijom (MPI)

```
static long num_steps = 100000;
void main(int argc, char* argv[])
{
   int i_start, i_end, i, myid, numprocs;
   double pi, mypi, x, step, sum = 0.0;
   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
   MPI_Comm_rank(MPI_COMM_WORLD, &myid);
   MPI_BCAST(&num_steps, 1, MPI_INT, 0,
                        MPI_COMM_WORLD);
   i_start = myid * (num_steps/numprocs)
   i_end = i_start + (num_steps/numprocs)
   step = 1.0 / (double) num_steps;
   for (i = i_start; i < i_end; i++) {
      x = (i + 0.5) * step
      sum = sum + 4.0 / (1.0 + x*x);
   }
   mypi = step * sum;
   MPI_REDUCE(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM,
                        0, MPI_COMM_WORLD);
   if (myid == 0)
      printf("Pi = %f\n", pi);
   MPI_Finalize();
}
```



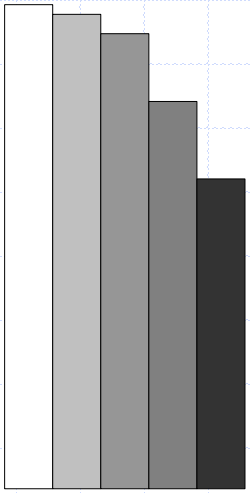
Računanje Pi sa integracijom (MPI)

```
static long num_steps = 100000;
void main(int argc, char* argv[])
{
   int i_start, i_end, i, myid, numprocs;
   double pi, mypi, x, step, sum = 0.0;
   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
   MPI_Comm_rank(MPI_COMM_WORLD, &myid);
   MPI_BCAST(&num_steps, 1, MPI_INT, 0,
                        MPI_COMM_WORLD);
   i_start = myid * (num_steps/numprocs)
   i_end = i_start + (num_steps/numprocs)
   step = 1.0 / (double) num_steps;
   for (i = i_start; i < i_end; i++) {
      x = (i + 0.5) * step
      sum = sum + 4.0 / (1.0 + x*x);
   }
   mypi = step * sum;
   MPI_REDUCE(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM,
                           0, MPI_COMM_WORLD);
   if (myid == 0)
      printf("Pi = %f\n", pi);
   MPI_Finalize();
}
```



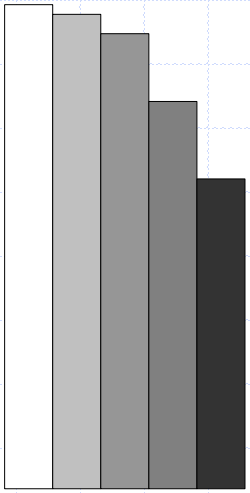
Računanje Pi sa integracijom (MPI)

```
static long num_steps = 100000;
void main(int argc, char* argv[])
{
   int i_start, i_end, i, myid, numprocs;
   double pi, mypi, x, step, sum = 0.0;
   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
   MPI_Comm_rank(MPI_COMM_WORLD, &myid);
   MPI_BCAST(&num_steps, 1, MPI_INT, 0,
                        MPI_COMM_WORLD);
   i_start = myid * (num_steps/numprocs)
   i_end = i_start + (num_steps/numprocs)
   step = 1.0 / (double) num_steps;
   for (i = i_start; i < i_end; i++) {
      x = (i + 0.5) * step
      sum = sum + 4.0 / (1.0 + x*x);
   }
   mypi = step * sum;
   MPI_REDUCE(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM,
                           0, MPI_COMM_WORLD);
   if (myid == 0)
      printf("Pi = %f\n", pi);
   MPI_Finalize();
}
```



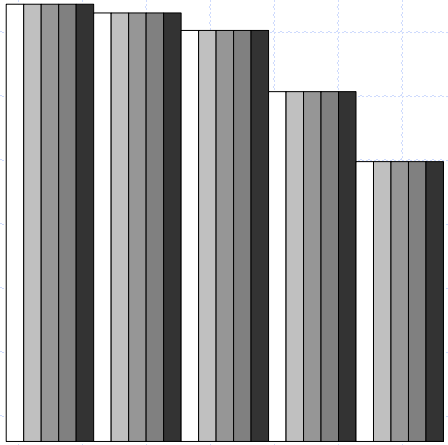
Računanje Pi sa integracijom (MPI)

```
static long num_steps = 100000;
void main(int argc, char* argv[])
{
   int i_start, i_end, i, myid, numprocs;
   double pi, mypi, x, step, sum = 0.0;
   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
   MPI_Comm_rank(MPI_COMM_WORLD, &myid);
   MPI_BCAST(&num_steps, 1, MPI_INT, 0,
                        MPI_COMM_WORLD);
   i_start = myid * (num_steps/numprocs)
   i_end = i_start + (num_steps/numprocs)
   step = 1.0 / (double) num_steps;
   for (i = i_start; i < i_end; i++) {
      x = (i + 0.5) * step
      sum = sum + 4.0 / (1.0 + x*x);
   }
   mypi = step * sum;
   MPI_REDUCE(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM,
                           0, MPI_COMM_WORLD);
   if (myid == 0)
      printf("Pi = %f\n", pi);
   MPI_Finalize();
}
```



Distribucija posla: Po bloku naspram Po ciklusu

```
static long num_steps = 100000;
void main(int argc, char* argv[])
{
   int i_start, i_end, i, myid, numprocs;
   double pi, mypi, x, step, sum = 0.0;
   MPI_Init(&argc, &argv);
   MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
   MPI_Comm_rank(MPI_COMM_WORLD, &myid);
   MPI_BCAST(&num_steps, 1, MPI_INT, 0,
                        MPI_COMM_WORLD);
   i_start = myid * (num_steps/numprocs)
   i_end = i_start + (num_steps/numprocs)
   step = 1.0 / (double) num_steps;
   for (i = myid; i < num_steps; i += numprocs) {
      x = (i + 0.5) * step
      sum = sum + 4.0 / (1.0 + x*x);
   }
   mypi = step * sum;
   MPI_REDUCE(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM,
                           0, MPI_COMM_WORLD);
   if (myid == 0)
      printf("Pi = %f\n", pi);
   MPI_Finalize();
}
```



Cilk jezik – kratak pregled

- Ključne reči za zadatke
  - cilk_spawn
  - cilk_sync
  - cilk_for
- Hiperobjekti (Hyperobjects)
  - Reduktori (reducers)
- Naznake za nizove
- SIMD (vektorske) funkcije
- SIMD pragma direktiva

Cilk jezik – duži pregled (1/3)

- Intel Cilk Plus je proširenje jezika C i C++
- Podržava ga raspoređivač zadataka
  - koji nije direktno izložen aplikacionom programeru
- Programeru su vidljivi
  - Tri ključne reči (spawn, sync i for)
  - Hiper promenljive – lokalni pogled na globalne promenljive
  - Naznake za nizove
  - Specijalne funkcije
  - Pragma SIMD
- CPU treba da ima više jezgara i vektorskih jedinica

Cilk jezik – duži pregled (2/3)

- Različito iskorišćenje komponenata CPU
  - spawn – koristi samo paralelizam jezgara
  - #pragma simd – koristi samo vektorske jedinice
  - Neke spcijalne funkcije koriste oba
- Serijalizacja = ponašanje Cilk programa
  - Isto kao ponašanje sličnih C/C++ programa
- Izvršenje
  - C/C++ programa = linearan niz iskaza
  - Cilk programa = usmeren acikličan graf

Cilk jezik – duži pregled (3/3)

- Paralelno izvršavanje može dovesti do trke do podataka
  - Delovi programa pristupaju podacima u nepoznatom redosledu
    - Bar jedan od pristupa je radi upisa (write access)

Model izvršenja zadataka u Cilku

- Radnik (worker)
  - Nit koja izvršava zadatke (obično: koliko procesora, toliko radnika)
- Linija izvršenja (strand)
  - Bilo koji niz instrukcija koje se izvršavaju serijski
- Tačke paralelizma
  - Tačka mrešćenja (*spawn*)
    - Jedna linija se pretvara u dve
  - Tačka sinhronizacije (sync)
    - Dve ili više linija se pretvaraju u jednu
    - Inicijalne linije se mogu izvršavati paralelno
    - Inicijalne linije se izvršavaju sekvencijalno sa novom linijom

Pravilo serijalizacije

- Cilk Plus program bez zadataka je C/C++ program sa istim ponašanjem
- Ako Cilk Plus program ima deterministično ponašanje, onda je ono isto za C/C++ program
  - Nakon uklanjanja svih instanci cilk_spawn i cilk_sync ključnih reči i
  - Nakon zamene svake instance cilk_for ključne reči sa for

Open MP – kratak pregled

- Glavne stvari se saopštavaju kroz pragme

  ...

Pragma

- Pragma je jedna od Ce/Ce++ pretprocesorskih direktiva.
- Ova direktiva predstavlja namenski napravljen mehanizam za pružanje dodatnih informacija kompajleru (koje ne mogu biti prenete kroz sam jezik).
- Ostavljeno je potpuno slobodno da se odredi šta će biti parametri direktive.

```
#pragma ovde može bilo šta: simboli, brojevi, stringovi...
```

- Postoje tri pragme koje se spominju u standardu. Sve ostale pragme koje budete susretali su specifične za platformu.
- Standard jedino propisuje da ako kompajler ne prepoznaje pragmu, onda samo treba da je ignoriše.
  - Pragma se može odnositi na celu datoteku (bez obzira gde je definisana), na datoteku od tačke definisanja pa na dalje (ili do, eventualno, mesta gde se drugom pragmom poništava važenje prve), samo na prvu narednu naredbu (ili red) itd. Zavisi od pragme.

Open MP – kratak pregled

- Glavne stvari se saopštavaju kroz pragme
- Sve pragme oblika

```
#pragma omp *
```

- OpenMP pragme se obično odnose na prvi sledeći „strukturirani blok“ (engl. structured block)
- Spominjaćemo sledeće pragme:
  - #pragma omp parallel
  - #pragma omp task
  - #pragma omp taskwait
  - #pragma omp parallel for
- Plus još neke stvari

Рачунање n-тог елемента у Фибоначијевом низу

```
Fib(n)
if n <= 1
  return n
else
  x = Fib(n - 1)
  y = Fib(n - 2)

  return x + y
```

Рачунање n-тог елемента у Фибоначијевом низу

```
Fib(n)
if n <= 1
  return n
else
  x = Fib(n - 1)
  y = Fib(n - 2)

  return x + y
```

```
Fib(n)
if n <= 1
  return n
else
  x = spawn Fib(n - 1)
  y = Fib(n - 2)
  sync
  return x + y
```

Рачунање n-тог елемента у Фибоначијевом низу

```
Fib(n)
if n <= 1
  return n
else
  x = Fib(n - 1)
  y = Fib(n - 2)

  return x + y
```

```
Fib(n)
if n <= 1
  return n
else
  x = spawn Fib(n - 1)
  y = Fib(n - 2)
  sync
  return x + y
```

Cilk Plus верзија

```
int Fib(int n) {
  if (n <= 1) return n;
  int x = cilk_spawn Fib(n - 1);
  int y = Fib(n - 2);
  cilk_sync;
  return x + y;
}
```

Mrešćenje zadatka (Spawn) (1/2)

- Sugeriše da se iskaz može izvršavati u paraleli sa sledećim iskazima
- Posledica je moguće nedefinisano ponašanje
  - koje ne postoji u sekvencijalnom izvršenju programa
- Izvršenje cilk_spawn se naziva *mrešćenje*
- Izvršenje cilk_sync se naziva *sinhronizacija*, sync
- Sledeći sync je onaj koji sledi u izvršenju u istom Cilk bloku

Mrešćenje zadatka (Spawn) (2/2)

- Sve operacije u izrazu za mrešćenje, koji ne moraju biti redne, obaviće se pre mrešćenja
- Linija koja započinje neposredno nakon tačke mrešćenja se naziva *nastavak* (iza mrešćenja)
- Niz operacija u iskazu mrešćenja je *potomak*
- Raspređivač može izvršiti potomka i nastavak u paraleli
- *Predak* je Cilk blok koji sadrži početnu liniju, iskaze mrešćenja i njihove nastavke, bez potomaka

Sync

- Sync iskaz u Cilk bloku označava da se svi potomci moraju završiti pre nastavka izvršenja
- Implicitni sync iskaz se nalazi na kraju funkcije u kojoj se nalazi spawn iskaz. (Ovakav implicitni sync se izvršava nakon svih destruktora)
- Ako se spawn pojavi u try bloku, implicitni sync je na kraju tog try bloka
- Ako nema potomaka u trenutku sync-a, on nema nikakvog efekta

Рачунање n-тог елемента у Фибоначијевом низу

OpenMP верзија

```
int Fib(int n) {
  if (n <= 1) return n;
#pragma omp task
  int x = Fib(n - 1);
  int y = Fib(n - 2);
#pragma omp taskwait
  return x + y;
}
```

Na šta se odnosi pragma? Šta je ovde strukturirani blok?

Рачунање n-тог елемента у Фибоначијевом низу

OpenMP верзија

```
int Fib(int n) {
  if (n <= 1) return n;
#pragma omp task
{
  int x = Fib(n - 1);
}
  int y = Fib(n - 2);
#pragma omp taskwait
  return x + y;
}
```

Iz perspektive OpenMP-a, kao da smo ovo napisali.

Рачунање n-тог елемента у Фибоначијевом низу

OpenMP верзија

```
int Fib(int n) {
  if (n <= 1) return n;
  int x;
#pragma omp task shared(x)
  x = Fib(n - 1);
  int y = Fib(n - 2);
#pragma omp taskwait
  return x + y;
}
```

Sada je dobro, ali i dalje moramo da naglasimo da li vrednost x kopiramo, ili želimo i da je menjamo.

Рачунање n-тог елемента у Фибоначијевом низу

OpenMP верзија

```
int Fib(int n) {
  if (n <= 1) return n;
  int x, y;
#pragma omp task shared(x)
  x = Fib(n - 1);
#pragma omp task shared(y)
  y = Fib(n - 2);
#pragma omp taskwait
  return x + y;
}
```

Често виђамо и овакав облик.

Рачунање n-тог елемента у Фибоначијевом низу

```
Fib(n)
if n <= 1
  return n
else
  x = Fib(n - 1)
  y = Fib(n - 2)

  return x + y
```

```
Fib(n)
if n <= 1
  return n
else
  x = spawn Fib(n - 1)
  y = Fib(n - 2)
  sync
  return x + y
```

OpenMP верзија

```
int Fib(int n) {
  if (n <= 1) return n;
  int x;
#pragma omp task shared(x)
  x = Fib(n - 1);
  int y = Fib(n - 2);
#pragma omp taskwait
  return x + y;
}
```

Cilk Plus верзија

```
int Fib(int n) {
  if (n <= 1) return n;
  x = cilk_spawn Fib(n - 1);
  y = Fib(n - 2);
  cilk_sync;
  return x + y;
}
```

cilk for petlje

- Cilk for petlja mora biti dobro formirana (imati kanončki oblik), tj. mora biti prava for petlja.
- To je suštinski ona petlja kod koje se broj iteracija zna u napred.
- Može se znati tokom prevođenja (veoma dobro formirana)
- Ili bar pre prve iteracije.

cilk_for

```
cilk_for (int i = 0; i < N; ++i)
  do_something(i);
```

Nije isto što i ovo:

```
for (int i = 0; i < N; ++i)
  cilk_spawn do_something(i);
cilk_sync;
```

Zašto?

cilk_for

```
cilk_for (int i = 0; i < N; ++i)
  do_something(i);
```

Već je kraći zapis za ovakav kod:

```
void recursion(int start, int end) {
  if (end - start <= G) {
    for (int i = start; i < end; ++i)
      do(i);
  }
  else {
    int middle = start + (end - start) / 2;
    cilk_spawn recursion(start, middle);
    recursion(middle, end);
    cilk_sync;
  }
}
recursion(0, N);
```

cilk_for

```
cilk_for (int i = 0; i < N; ++i)
  do_something(i);
```

Već je kraći zapis za ovakav kod:

```
void recursion(int start, int end) {
  if (end - start <= min(2048, N / (8*p))) {
    for (int i = start; i < end; ++i)
      do(i);
  }
  else {
    int middle = start + (end - start) / 2;
    cilk_spawn recursion(start, middle);
    recursion(middle, end);
    cilk_sync;
  }
}
recursion(0, N);
```

cilk_for

```
#pragma cilk grainsize = 100 // min(2048, N / (8*p))
cilk_for (int i = 0; i < N; ++i)
  do_something(i);
```

Već je kraći zapis za ovakav kod:

```
void recursion(int start, int end) {
  if (end - start <= 100) {
    for (int i = start; i < end; ++i)
      do(i);
  }
  else {
    int middle = start + (end - start) / 2;
    cilk_spawn recursion(start, middle);
    recursion(middle, end);
    cilk_sync;
  }
}
recursion(0, N);
```

OpenMP parallel for

```
#pragma omp parallel for
for (int i = 0; i < N; ++i)
  do_something(i);
```

```
#pragma omp parallel
#pragma omp for
for (int i = 0; i < N; ++i)
  do_something(i);
```

Jeste skraćeni oblik zapisa ovoga:

```
#pragma omp parallel
{
  int id = omp_get_thread_num();
  int Nthrds = omp_get_num_threads();
  int istart = id * N / Nthrds;
  int iend = (id+1) * N / Nthrds;
  if (id == Nthrds-1) iend = N;
  for (int i = istart; i < iend; ++i) {
    do_something(i);
  }
}
```

Reduktori

```
Mat-Vec-Wrong(A, x)
  n = A.rows
  neka je y novi vector dužine n
  parallel for i = 1 to n
    y_i = 0
  parallel for i = 1 to n
    for j = 1 to n
      y_i = y_i + a_ij x_j
  return y
```

```
vector<int> MatVec(matrix<int> A, vector<int> x) {
  int n = A.rows();
  vector<int> y(x.size());

  for (int i = 0; i < n; ++i)
    for (int j = 0; j < n; ++j)
      y[i] += a(i,j) * x[j];
  return y;
}
```

Reduktori

```
vector<int> MatVec(matrix<int> A, vector<int> x) {
  int n = A.rows();
  vector<int> y(x.size());

  cilk_for (int i = 0; i < n; ++i)
    cilk_for (int j = 0; j < n; ++j)
      y[i] += a(i,j) * x[j];
  return y;
}
```

```
vector<int> MatVec(matrix<int> A, vector<int> x) {
  int n = A.rows();
  vector<int> y(x.size());

  #pragma omp parallel for
  for (int i = 0; i < n; ++i)
    #pragma omp parallel for
    for (int j = 0; j < n; ++j)
      y[i] += a(i,j) * x[j];
  return y;
}
```

Reduktori Cilk Plus

```
#include <cilk/reducer_opadd.h>
using namespace cilk;

vector<int> MatVec(matrix<int> A, vector<int> x) {
  int n = A.rows();
  vector<reducer_oppadd<int>> t(x.size());
  vector<int> y(x.size());

  cilk_for (int i = 0; i < n; ++i) {
    cilk_for (int j = 0; j < n; ++j)
      t[i] += A(i,j) * x[j];
    y[i] = t[i].get_value();
  }

  return y;
}
```

Reduktori OpenMP

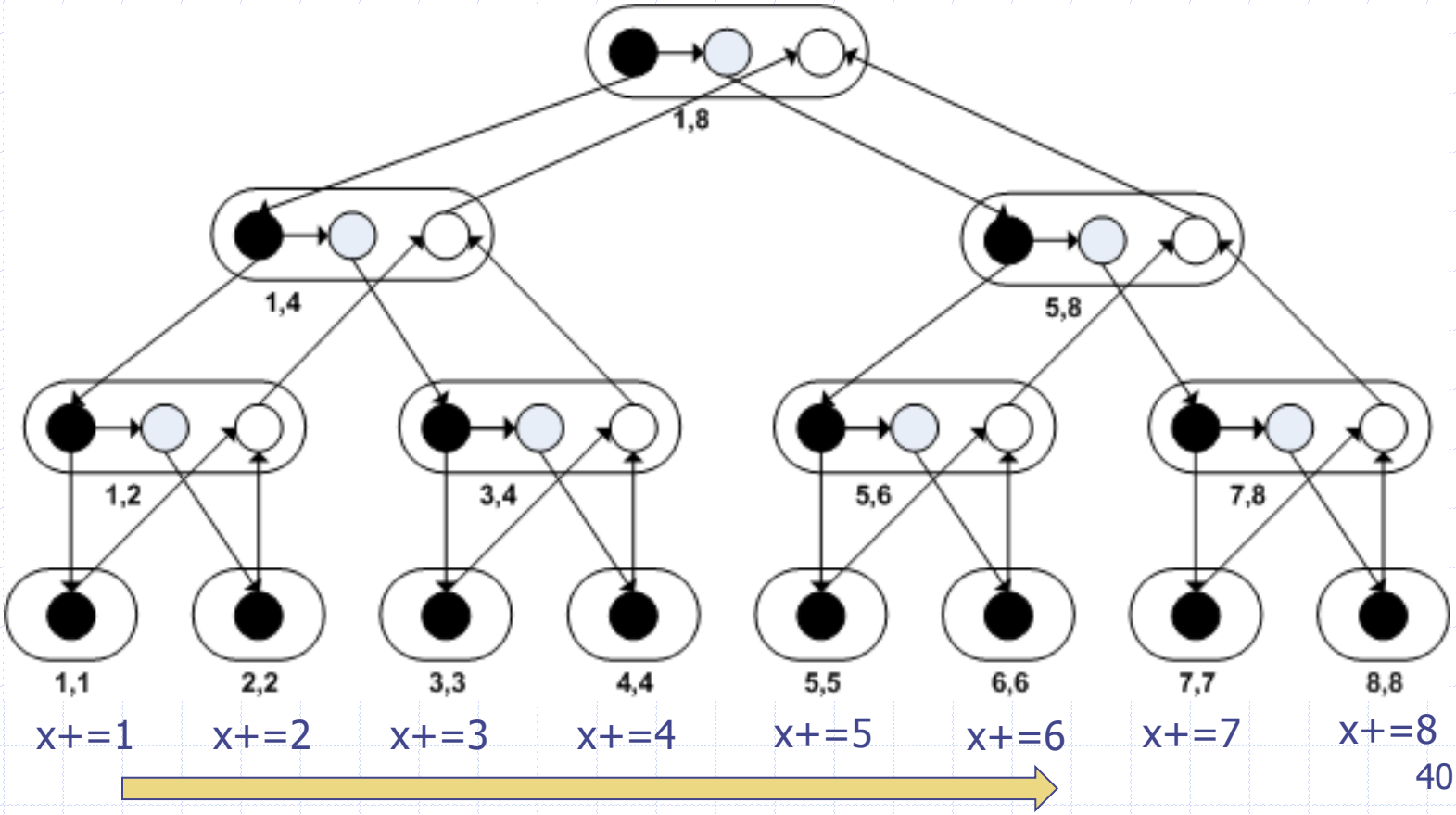
```
vector<int> MatVec(matrix<int> A, vector<int> x) {
  int n = A.rows();
  int* t = new int[x.size()];
  vector<int> y(x.size());

  #pragma omp parallel for
  for (int i = 0; i < n; ++i) {
    #pragma omp parallel for reduction(+:t[i])
    for (int j = 0; j < n; ++j)
      t[i] += A(i,j) * x[j];
    y[i] = t[i];
  }

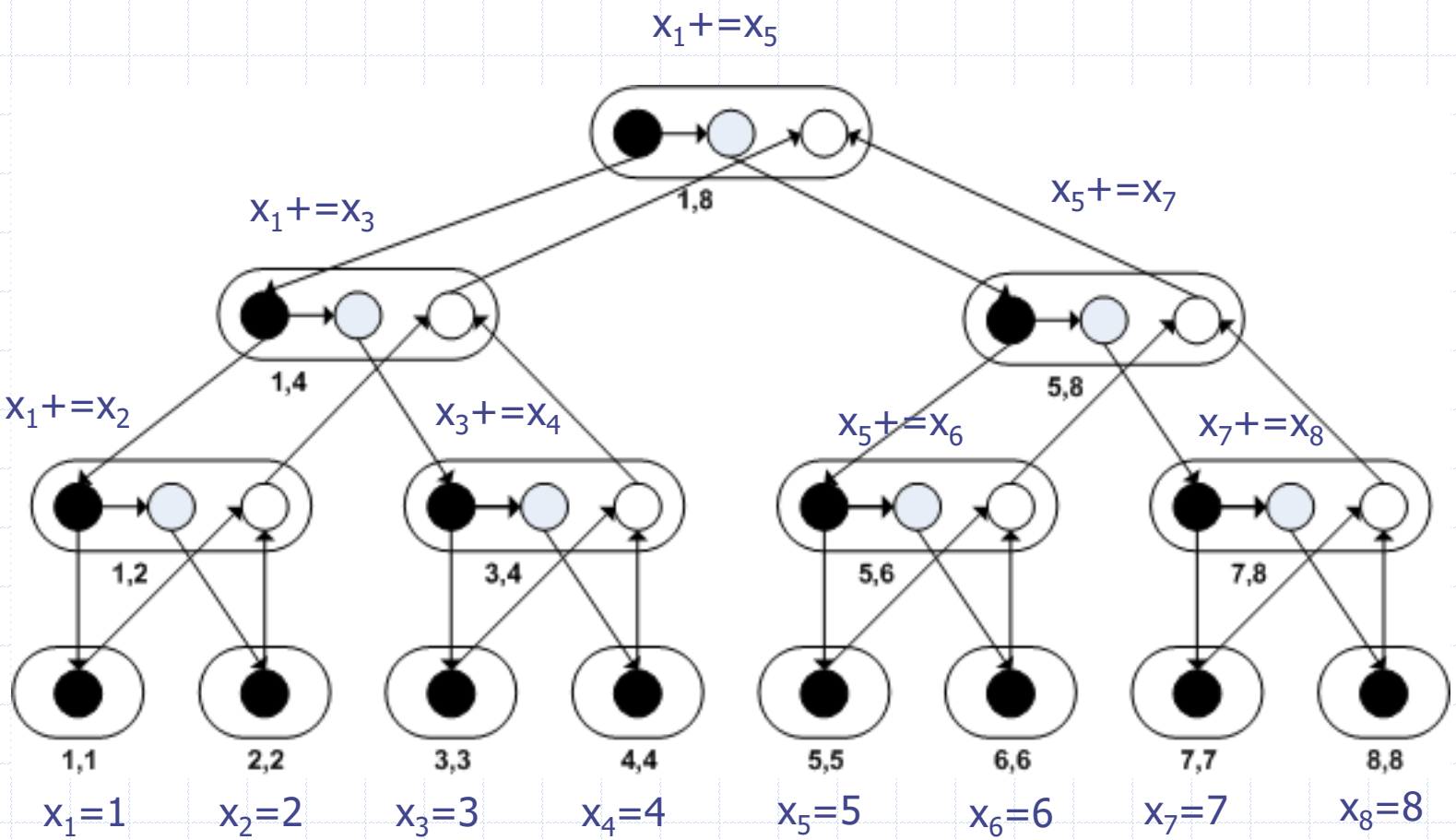
  delete[] t;

  return y;
}
```

Reduktori



Reduktori


$x_1+=x_5$
1,8
$x_1+=x_3$
$x_5+=x_7$
1,4
5,8
$x_1+=x_2$
$x_3+=x_4$
$x_5+=x_6$
$x_7+=x_8$
1,2
3,4
5,6
7,8
1,1
2,2
3,3
4,4
5,5
6,6
7,7
8,8
$x_1=1$
$x_2=2$
$x_3=3$
$x_4=4$
$x_5=5$
$x_6=6$
$x_7=7$
$x_8=8$

Hiperobjekti

- Omogućavaju siguran pristup deljenim objektima dajući svakoj paralelnoj liniji posebnu instancu
- Obraćanje hiperobjektu rezultuje u referenci
  - koja se naziva ***pogled***
    - Pogled se stvara pozivom funkcije tipa hiperobjekta
    - Pogledi se spajaju, pri sync, u drugoj funkciji
    - Identitet (adresa) pogleda u jednoj liniji se ne menja
    - Pogled pre spawn i nakon sync je isti
      - Mada ID programske niti (thread) ne mora biti isti
    - Pogled pre i posle cilk_for je isti
    - Specijalan, najraniji, pogled se stvara pri stvaranju hiperobjekta

Reduktori

- Hiperobjekti najčešće spadaju u reduktore
- Tip reduktora definiše operaciju *reduce*
  - koja spaja dva pogleda svojstveno reduktoru
  - reduce(V1, V2) se označava kao V1◦V2
  - Klasična reduce operacija je asocijtivna (a◦b)◦c == a◦(b◦c)
- Definiše i operaciju identity
  - koja inicijalizuje novi pogled, I
  - I◦v == v i v◦I == v, za bilo koji v tipa value_type
- Trojka (◦,I,value_type) opisuje matematicki *monoid*

Reduktori

- Monoidi: (int,+,0), (list,concatenate,empty)...
- Ako se svaki pogled reduktora modifikuje isključivo operacijama $R \leftarrow R \circ v$
  - gde je v tipa value_type
  - Reduktor dolazi do istog rezultata u paralelnom programu, kao da je on serijalizovan
  - Operacija $\circ$ može odgovarati skupu operacija, npr. +=, -= su asocijativne.
  - Npr. telo cilk_for petlje može dodavati elemente na kraj reduktor liste – rezultantna lista je ista kao ona koja bi se generisala serijskim izvršenjem

Reduktori

- Kada niz linija S1 S2... Sn sa pogledima V1 V2... Vn ulazi u sync, rezultat je
  - jedan pogled W ← V1◦V2◦ ... ◦Vn
  - održava se redosled s-leva-na-desno
  - grupisanje operacija (asocijativnost) nije poznato
  - dinamika ove redukcije nije poznata
- Ako reduce nije asocijativna ili identity ne vraća pravi identitet, rezutat je nedeterministič an
- Reduce operacija ne mora biti komutativna

Reduktori

- Pravljenje pogleda nije besplatno
- Novi pogleda je potrebno praviti samo ako linija izvršenja pristupa hiperobjektu i ako se zaista izvršava paralelno sa nekom drugom linijom.
- Zbog toga se novi pogleda pravi akko dođe do preuzimanja zadatka od strane drugog „radnika“ (procesora) i to tek kada se dogodi prvi pristup pogledu u tom preuzetom zadatku.

OpenMP

Cilk Plus	OpenMP
cilk_spawn	#pragma omp task
cilk_sync	#pragma omp taskwait
cilk_for	#pragma omp parallel for
#pragma simd	#pragma omp simd
__declspec(vector()), __attribute__((vector()))	#pragma omp declare simd